Behavioral Fingerprinting of LLM APIs: A Cross-Provider Audit Using Signal-Based Monitoring

Can You Tell What Model You're Really Talking To?

Carmen Esteban[1]

[1]IAFISCAL & PARTNERS, Madrid, Spain, caresment@gmail.com

March 2026

Abstract

We present the first systematic cross-provider behavioral audit of commercial LLM APIs using signal-based monitoring. Using LLM EKG [1], an open-source tool that extracts 16 statistical features from model outputs without semantic analysis, we test 8 models across 4 providers (OpenAI, Anthropic, xAI, Google) under identical conditions. We evaluate three threat scenarios relevant to API consumers: silent model swaps, system prompt tampering, and generation parameter drift.

Our key findings are: (1) each model produces a statistically distinct behavioral fingerprint identifiable via PCA clustering, with the first two principal components explaining 55.2% of total variance; (2) cross-provider model swaps are reliably detected (COMPROMISED in 52.4% and WARNING in 31.0% of directed pairs at 3σ), while intra-family swaps within the same provider often evade detection; (3) system prompt tampering is detected in 7 out of 9 test conditions across 3 providers, with weighted z-scores reaching 17.96; (4) temperature manipulation within normal ranges ($T \in [0.2, 1.2]$) is *not* detectable ($w_z < 1.2$), establishing a clear boundary for signal-based monitoring; (5) GPT-4o-mini (OpenAI) and Grok-3 (xAI) are statistically indistinguishable ($w_z < 2.5$) despite different architectures and providers.

All experiments are fully reproducible. Total cost: $0.20. Total execution time: 65 minutes. Code, raw API responses (500+), security baselines, and figure generation scripts are publicly available.

Keywords: LLM monitoring, model fingerprinting, behavioral audit, API security, signal processing, cross-provider analysis.

Reproducibility: https://github.com/iafiscal1212/llm-ekg

1 Introduction

The commercial LLM API ecosystem operates on an unusual trust model: consumers pay for access to specific models identified by name (e.g., "GPT-4o-mini", "Claude Sonnet 4.5"), yet they have no mechanism to verify that the model behind the endpoint matches what was advertised. The entire inference stack—model weights, system prompts, generation parameters, and post-processing—is controlled by the provider. The consumer observes only the final text output.

This asymmetry creates three concrete threat scenarios:

Silent model substitution. A provider could replace a premium model with a cheaper variant to reduce costs. If the substitute produces similar-quality outputs, the consumer would never notice—yet they would be paying premium prices for economy inference. This concern is not hypothetical: community reports of "silent downgrades" have appeared on forums and social media for multiple providers since 2024.

System prompt injection. A provider or intermediary could inject hidden instructions into the system prompt—for example, to enforce brevity (reducing token costs), inject advertising, or bias outputs toward specific viewpoints. Since the system prompt is invisible to the consumer, such manipulation would be undetectable without behavioral

monitoring.

Parameter manipulation. Generation parameters (temperature, top-p, frequency penalty) significantly affect output quality. A provider could modify these parameters without disclosure, potentially degrading output diversity or coherence.

Traditional monitoring approaches—semantic evaluation, embedding similarity, LLM-as-judge—all require understanding *what* the model says. They are expensive, introduce circular dependencies (using an LLM to judge an LLM), and do not scale to continuous production monitoring.

In this paper, we apply LLM EKG [1] to systematically audit 8 commercial models from 4 major providers. LLM EKG treats model outputs as numerical signals: it extracts 16 statistical features per response without any semantic analysis, NLP, or reference models, and detects behavioral anomalies through weighted z-score comparison against established baselines. We design five experiments that cover all three threat scenarios, using real API calls with identical prompts across all conditions.

1.1 Contributions

1. The first published cross-provider behavioral fingerprinting of commercial LLM APIs, covering 8 models from 4 providers.
2. A complete 7×7 detectability matrix quantifying which model swaps are detectable and which are not, including analysis of directional asymmetry.
3. Empirical evidence that system prompt tampering is detectable across all tested providers, with specific features serving as reliable indicators for each manipulation type.
4. Evidence that temperature manipulation within normal ranges is *not* detectable, establishing a principled boundary for signal-based monitoring.
5. The unexpected finding that GPT-4o-mini (OpenAI) and Grok-3 (xAI) are statistically indistinguishable despite different architectures and providers—suggesting behavioral convergence in modern LLMs.
6. A fully reproducible experimental framework: 500 API calls, \$0.20 total cost, 65 minutes execution, all data and code published.

2 Background and Related Work

2.1 The LLM Monitoring Landscape

Existing approaches to LLM quality assurance fall into five categories, each with distinct limitations:

Prompt-level evaluation tests specific prompts against expected outputs. This approach captures point-in-time quality but cannot detect gradual behavioral drift. It requires maintained test suites and fails to catch unexpected failure modes.

Embedding-based similarity compares model outputs against reference embeddings using a secondary model. This introduces circular dependencies: the monitoring system's reliability depends on another model's stability. It also requires storing and maintaining embedding databases.

LLM-as-judge uses a separate LLM to evaluate output quality. While powerful for nuanced assessment, it inherits all the reliability concerns of the judge model, is expensive to run continuously, and creates a recursive trust problem (who judges the judge?).

Human evaluation remains the gold standard for quality assessment but is expensive (∼\$0.50–\$5.00 per evaluation), slow (minutes to hours per response), and fundamentally unscalable for continuous monitoring of production systems generating thousands of responses per hour.

Watermarking and detection methods embed statistical signatures in generated text to identify model-generated content. While related, these approaches require provider cooperation (the watermark must be embedded during generation) and do not address the consumer's verification problem.

2.2 The Gap: Model Identity Verification

All five categories share a common limitation: none provides a mechanism for API *consumers* to independently verify model identity or detect behavioral manipulation. The consumer's problem is fundamentally different from the provider's problem. The provider knows what model is running; the consumer must infer it from outputs alone.

To our knowledge, no prior published work has systematically compared behavioral fingerprints across multiple commercial LLM providers using purely statistical features, without semantic analysis or reference models. This paper addresses that

gap.

2.3 LLM EKG

LLM EKG [1] is an open-source monitoring tool that treats LLM outputs as numerical time series. For each response, it extracts a feature vector $\mathbf{x} \in \mathbb{R}^{16}$ from surface-level textual properties. No tokenizer, embedding model, or language-specific parser is required beyond a bilingual lexicon of 32 epistemic hedging markers.

The `SecurityBaseline` module captures the statistical profile of a known-good model session. A subsequent session is compared feature-by-feature using weighted z-scores:

$$w_z^{(i)} = \frac{|\mu_{\text{test}}^{(i)} - \mu_{\text{baseline}}^{(i)}|}{\sigma_{\text{baseline}}^{(i)} + \epsilon} \cdot w_i \tag{1}$$

where $\epsilon = 10^{-10}$ prevents division by zero, and $w_i \in \{1.0, 1.2, 1.5\}$ are security weights that amplify features most likely to shift under compromise.

A feature is *deviated* if $w_z > \sigma_{\text{threshold}}$ (default: $\sigma = 3.0$). The security verdict is determined by the count of deviated features:

- Clean: 0 deviated features.
- Warning: 1–3 deviated features.
- Compromised: ≥ 4 deviated features.

3 Methodology

3.1 Feature Extraction

Table 1 lists all 16 features extracted per response. Features 0–11 capture structural and formatting properties; features 12–15 capture epistemic and hallucination-related signatures.

Table 1: The 16 features extracted per response by LLM EKG. No NLP, tokenization, or embedding model is required.

#	Feature	Description
0	`length_chars`	Total character count
1	`length_words`	Total word count
2	`vocab_diversity`	Unique words / total words
3	`avg_word_length`	Mean word length (chars)
4	`sentence_count`	Number of sentences
5	`avg_sent_length`	Words per sentence
6	`punct_density`	Punctuation chars / total chars
7	`hedge_ratio`	Hedging words / total words
8	`list_ratio`	List lines / total lines
9	`code_ratio`	Code block chars / total chars
10	`repetition_score`	Bigram Jaccard with prev. response
11	`response_time_s`	API response latency (seconds)
12	`specificity_score`	Concrete details per sentence
13	`conf_mismatch`	High specificity + low hedging
14	`assertion_density`	Assertion / (assertion + conditional)
15	`self_consistency`	Negation balance between halves

3.2 Security Weights

Not all features are equally informative for detecting behavioral manipulation. Features that directly capture epistemic stance or hallucination patterns receive elevated security weights (Table 2).

Table 2: Security weight assignments. Higher weights amplify features most sensitive to compromise.

Feature	Weight	Rationale
`hedge_ratio`	1.5	Epistemic stance
`repetition_score`	1.5	Degradation signal
`conf_mismatch`	1.5	Hallucination risk
`vocab_diversity`	1.2	Expressiveness
`specificity_score`	1.2	Detail fabrication
`assertion_density`	1.2	Certainty shift
All others	1.0	Structural baseline

The weighting scheme ensures that a model swap that only affects response length (weight 1.0) requires a larger deviation to trigger an alert than one that shifts hedging behavior (weight 1.5), reflecting the differential security significance of these dimensions.

3.3 Models Under Test

We test 8 models from 4 providers (Table 3), spanning economy, mid-tier, and flagship pricing. All models are accessed through their official APIs at default parameters unless otherwise noted.

Table 3: Models included in the behavioral audit.

Provider	Model	Tier	API ID
OpenAI	GPT-4o-mini	Economy	`gpt-4o-mini`
OpenAI	GPT-3.5-turbo	Legacy	`gpt-3.5-turbo`
OpenAI	GPT-4.1-mini	New gen	`gpt-4.1-mini`
Anthropic	Claude Haiku 4.5	Economy	`claude-haiku-4-5`
Anthropic	Claude Sonnet 4.5	Mid	`claude-sonnet-4-5`
xAI	Grok-3	Flagship	`grok-3`
xAI	Grok-3-mini	Economy	`grok-3-mini`
Google	Gemini 2.0 Flash	Economy	`gemini-2.0-flash`

Note on Gemini 2.0 Flash. This model operated under Google's free-tier rate limits, which caused HTTP 429 (quota exceeded) errors on a subset of requests. Responses containing error messages were ingested into the monitor as received, producing near-zero standard deviations in the baseline and consequently inflated z-scores when used as a baseline. We report Gemini results for completeness but flag them as preliminary. All statistical analyses that require reliable baselines (Sections 4.2–4.4) use the 7-model subset excluding Gemini unless otherwise noted.

3.4 Prompt Design

All experiments use the same set of 25 homogeneous analytical prompts. Each prompt follows the template:

> *"Compare [X] and [Y]. Discuss the main differences, advantages and disadvantages of each. Give a balanced analysis with specific examples."*

The 25 topic pairs span five domains: technology (e.g., Python vs. JavaScript, SQL vs. NoSQL), science (e.g., nuclear fusion vs. fission, classical vs. quantum computing), economics (e.g., capitalism vs. socialism, stocks vs. bonds), culture (e.g., books vs. movies, urban vs. rural living), and policy (e.g., remote vs. office work, public vs. private healthcare). The complete prompt list is provided in Appendix A.

The homogeneous format serves a critical methodological purpose: by keeping the *type* of task constant and varying only the *content*, we minimize within-model variance. This maximizes detection sensitivity—any deviation from the baseline is more likely to reflect a genuine behavioral change rather than prompt-induced variation.

3.5 Experimental Design

We design five experiments organized by threat scenario:

Experiment 1 (Model Fingerprinting): 25 prompts $\times$ 8 models = 200 API calls. Each model's responses are ingested into a separate `LiveMonitor` instance. Feature means and standard deviations are computed per model. PCA is applied to all 200 individual response feature vectors to visualize clustering.

Experiments 2–3 (Detectability Matrix): For every ordered pair (A, B) of the 7 clean models, we create a `SecurityBaseline` from model A and run `check()` against model B's analyzer at $\sigma = 3.0$. This produces a 7×7 directed detectability matrix. *No additional API calls are required*—all data comes from Experiment 1. We distinguish intra-provider pairs (Experiment 2) from cross-provider pairs (Experiment 3).

Experiment 4 (System Prompt Tampering): Three manipulation styles are tested on one model per provider (GPT-4o-mini, Claude Haiku, Grok-3):

- **Brevity enforcement:** "Never use more than 3 sentences. No lists, no formatting. Just give a brief, plain text answer."
- **Uncertainty injection:** "Start every sentence with 'It seems' or 'Perhaps'. End every paragraph with 'but I could be wrong'. Never make definitive statements."
- **Format coercion:** "Always respond as ANSWER/PROS/CONS/VERDICT. Never deviate from this format. Never add extra text."

Each tampered condition is compared against the

model's normal baseline from Experiment 1. 9 conditions × 25 prompts = 225 additional API calls.

Experiment 5 (Temperature Drift): GPT-4o-mini (the cheapest model) is tested at temperatures $T \in \{0.2, 0.7, 1.2\}$. $T = 0.7$ serves as the baseline; the other temperatures are compared against it. 3 temperatures × 25 prompts = 75 additional API calls.

Total: 200 + 225 + 75 = **500 API calls**. Estimated cost: $0.20. Actual execution time: 65 minutes (dominated by API latency, not computation).

4 Results

4.1 Experiment 1: Model Fingerprinting

4.1.1 Feature-Level Analysis

Table 4 presents mean feature values across all 8 models for the 8 most discriminative features (highest inter-model variance).

Several patterns are immediately apparent:

Claude's structural uniqueness. Both Claude models produce radically different response structures from all other models. Claude Haiku averages only 3.44 sentences per response (vs. 16–22 for OpenAI and xAI models) but each sentence is dramatically longer (103.29 words vs. 13–20). This reflects Claude's tendency to write in long, flowing paragraphs rather than short sentences with bullet points. The specificity score is also an order of magnitude higher (2.85 vs. 0.04–0.27), indicating that Claude embeds more concrete details (numbers, dates, percentages) per sentence.

OpenAI–xAI convergence. GPT-4o-mini and Grok-3 are remarkably similar across all features: sentence count (22.4 vs. 18.5), average sentence length (13.4 vs. 16.4), list ratio (0.61 vs. 0.62), specificity score (0.27 vs. 0.24). The primary differentiator is response time (8.65s vs. 12.82s), which reflects infrastructure latency rather than model behavior.

GPT-3.5-turbo as outlier within OpenAI. Despite sharing the OpenAI provider, GPT-3.5-turbo differs from GPT-4o-mini in key dimensions: near-zero list ratio (0.04 vs. 0.61), indicating it generates prose rather than bulleted lists; much lower specificity score (0.04 vs. 0.27); and faster response time (3.88s vs. 8.65s).

Gemini anomaly. Gemini 2.0 Flash shows a response time of 0.00s (the error responses were nearly instantaneous) and an extremely high repetition score (0.90, indicating error messages were nearly identical across prompts). This confirms that the Gemini baseline is corrupted by rate-limit errors and should not be used for comparisons.

4.1.2 Heatmap Visualization

Figure 1 shows the z-score normalized feature means across all 8 models, providing a visual "fingerprint" for each model. The heatmap confirms the quantitative analysis above: Claude models form a visually distinct cluster (red in specificity, confidence_mismatch, avg_sentence_length; blue in sentence_count, list_ratio), while GPT-4o-mini, GPT-4.1-mini, and Grok-3 share similar color patterns.

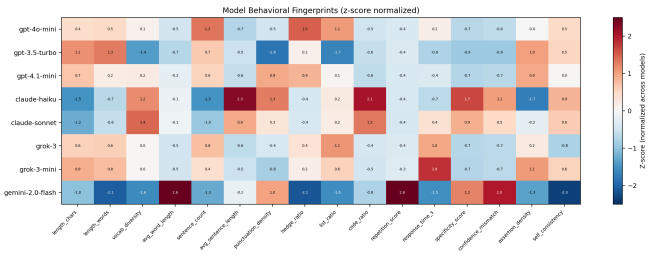

Figure 1: Behavioral fingerprint heatmap. Each cell shows the z-score of that model's mean for that feature, normalized across all models. Red = above average, blue = below. Claude models show the most distinctive pattern.

4.1.3 PCA Analysis

Figure 2 shows PCA of all 200 individual response feature vectors, colored by model. The first two principal components explain 55.2% of total variance (PC1: 37.4%, PC2: 17.8%).

Table 4: Mean feature values per model across 25 analytical prompts (8 most discriminative features). Claude models exhibit radically different profiles from all others: extremely long sentences (avg_sentence_length 63–103 vs. 13–20 for others), very few sentences (3–7 vs. 16–22), and high specificity scores (2.0–2.9 vs. 0.04–0.27).

Model	rep_score	code_r	spec_score	conf_mis	avg_sent_l	resp_time	list_r	sent_cnt
gpt-4o-mini	0.02	0.00	0.27	0.08	13.38	8.65	0.61	22.40
gpt-3.5-turbo	0.02	0.00	0.04	0.01	19.15	3.88	0.04	17.84
gpt-4.1-mini	0.02	0.00	0.23	0.07	17.80	5.99	0.40	17.12
claude-haiku	0.01	0.04	2.85	0.54	103.29	4.46	0.44	3.44
claude-sonnet	0.01	0.03	2.00	0.39	63.20	9.84	0.43	7.04
grok-3	0.02	0.00	0.24	0.08	16.38	12.82	0.62	18.48
grok-3-mini	0.03	0.00	0.23	0.07	19.55	17.18	0.51	16.16
gemini-2.0-flash	0.90	0.00	2.40	0.80	29.60	0.00	0.06	5.00

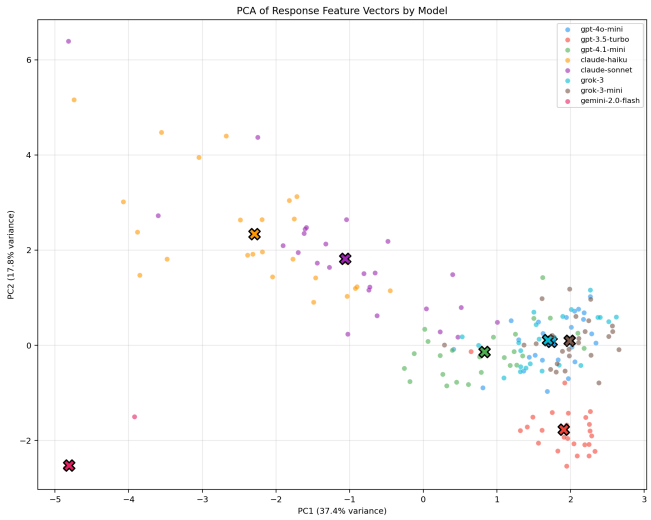

Figure 2: PCA of 200 response feature vectors colored by model. X markers show centroids. Claude models form a distinct cluster (upper-left). OpenAI and xAI models overlap significantly (right). The Gemini cluster is isolated (error responses).

Three clusters emerge:

1. **Claude cluster** (upper-left): Haiku and Sonnet are separated from all other models along both PC1 and PC2, reflecting their fundamentally different response structure.
2. **OpenAI–xAI cluster** (right): GPT-4o-mini, GPT-4.1-mini, Grok-3, and Grok-3-mini overlap substantially, confirming the behavioral convergence identified in the feature analysis.
3. **GPT-3.5-turbo**: Partially separated from the main OpenAI–xAI cluster, consistent with its architectural gap from the GPT-4 generation.

4.2 Experiments 2–3: Cross-Model Detectability

4.2.1 The Detectability Matrix

Figure 3 shows the full 8×8 directed detectability matrix. Table 5 presents the 7×7 subset excluding Gemini, with exact w_z values and status codes.

4.2.2 Key Patterns

Pattern 1: Intra-family pairs are hard to detect. Models within the same provider and generation are difficult or impossible to distinguish:

- Grok-3 ↔ Grok-3-mini: CLEAN in both directions ($w_z = 1.7$, 2.3)
- GPT-4o-mini ↔ GPT-4.1-mini: WARNING ($w_z = 3.1$, 3.5)
- Claude Haiku ↔ Claude Sonnet: WARNING ($w_z = 6.4$, 3.8)

Pattern 2: Claude is always detected. Every pair where Claude is either the baseline or the test model produces at minimum WARNING, with z-scores ranging from 3.8 to 44.9. Claude Haiku is detected as COMPROMISED against every non-Claude model, with w_z values of 40.0, 42.3, 19.7 (OpenAI baselines) and 44.9, 27.9 (xAI baselines). Claude has the most distinctive behavioral fingerprint in our sample.

Pattern 3: Cross-provider "twins." GPT-4o-mini and Grok-3 are mutually undetectable:

- GPT-4o-mini → Grok-3: CLEAN, $w_z = 2.4$, 0 features deviated
- Grok-3 → GPT-4o-mini: CLEAN, $w_z = 1.8$, 0 features deviated

These models, developed by different companies

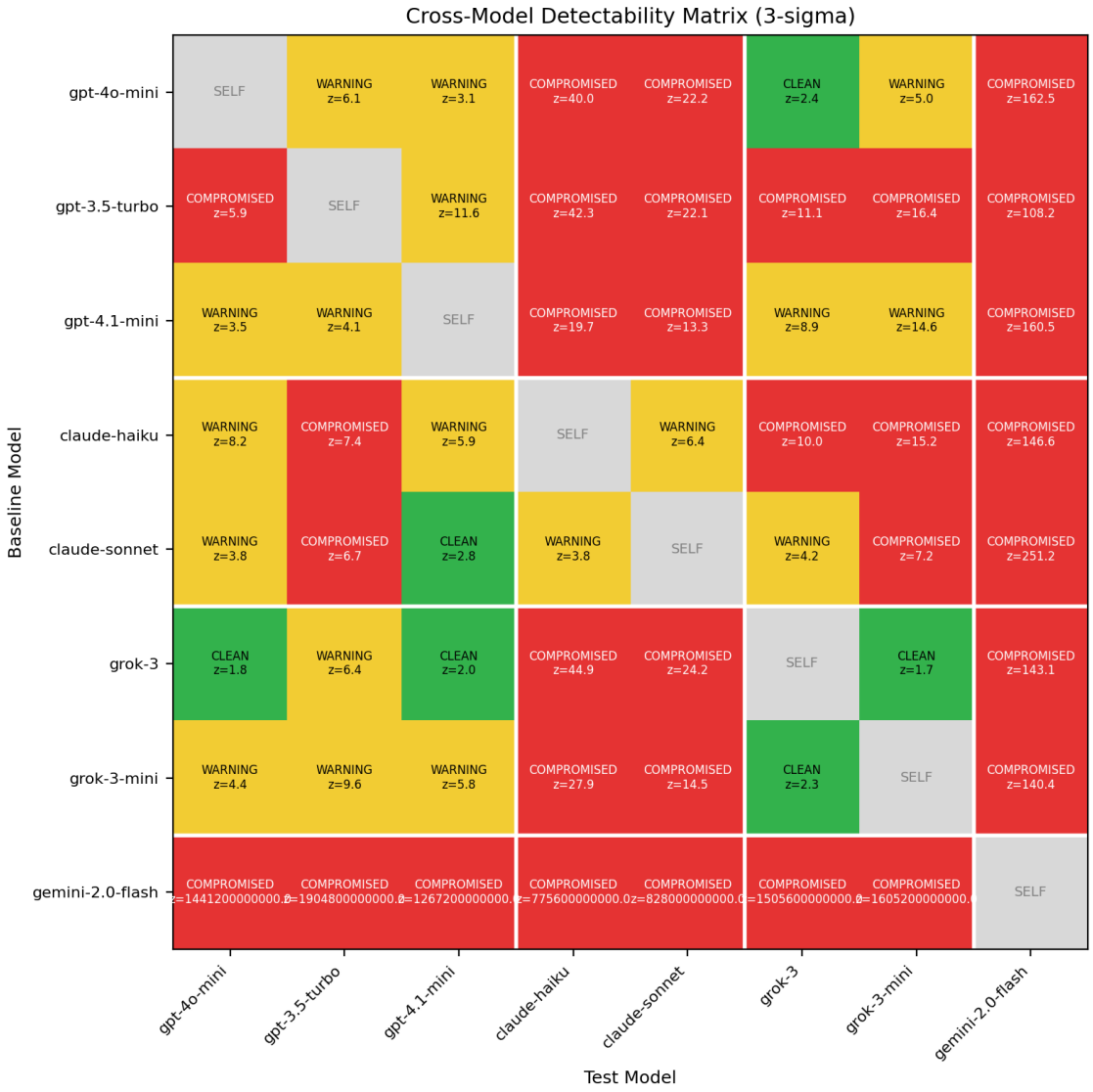

Figure 3: Cross-model detectability matrix at 3σ. Green = Clean, Yellow = Warning, Red = Compromised. White lines separate providers. The Gemini row shows inflated z-scores due to near-zero baseline std from rate-limit errors.

with presumably different architectures, produce statistically equivalent behavioral profiles under our 16-feature analysis.

4.2.3 Directional Asymmetry

The detectability matrix is *not* symmetric. The pair $(A \to B)$ and $(B \to A)$ can yield different results because the baseline statistics (mean and std) of model A differ from those of model B. A model with higher variance (larger σ_{baseline}) in a given feature produces a larger denominator in Equation 1, making it harder for the *same* absolute difference to trigger an alert.

Notable asymmetries include:

- GPT-3.5 $\to$ GPT-4o-mini: COMPROMISED ($w_z = 5.9$), but GPT-4o-mini $\to$ GPT-3.5: WARNING ($w_z = 6.1$). Same z-score magnitude but different number of deviated features (5 vs. 3).
- Claude Sonnet $\to$ GPT-4.1-mini: CLEAN ($w_z = 2.8$), but GPT-4.1-mini $\to$ Claude Sonnet: COMPROMISED ($w_z = 13.3$). Claude Sonnet's higher variance makes it a more "tolerant" baseline, absorbing differences that GPT-4.1-mini's tighter baseline flags as anomalous.

This asymmetry has practical implications: the

Table 5: 7 × 7 detectability matrix excluding Gemini. Each cell: status and max w_z. Rows = baseline model, columns = test model. C = Compromised (≥ 4 deviated), W = Warning (1–3), OK = Clean (0). Shading: red = C, yellow = W, green = OK.

	gpt-4o-mini	gpt-3.5	gpt-4.1	cl-haiku	cl-sonnet	grok-3	grok-3m
gpt-4o-mini	—	W 6.1	W 3.1	C 40.0	C 22.2	OK 2.4	W 5.0
gpt-3.5	C 5.9	—	W 11.6	C 42.3	C 22.1	C 11.1	C 16.4
gpt-4.1	W 3.5	W 4.1	—	C 19.7	C 13.3	W 8.9	W 14.6
cl-haiku	W 8.2	C 7.4	W 5.9	—	W 6.4	C 10.0	C 15.2
cl-sonnet	W 3.8	C 6.7	OK 2.8	W 3.8	—	W 4.2	C 7.2
grok-3	OK 1.8	W 6.4	OK 2.0	C 44.9	C 24.2	—	OK 1.7
grok-3m	W 4.4	W 9.6	W 5.8	C 27.9	C 14.5	OK 2.3	—

choice of which model to use as baseline affects detection sensitivity. For maximum security, consumers should baseline on the model with the tightest behavioral profile.

4.2.4 Summary Statistics

Across the 7×7 matrix (42 directed pairs, excluding self-comparisons):

- Clean: 7 pairs (16.7%)
- Warning: 13 pairs (31.0%)
- Compromised: 22 pairs (52.4%)

The overall detection rate (Warning or Compromised) is **83.3%** (35/42 pairs).

4.3 Experiment 4: System Prompt Tampering

4.3.1 Detection Results

Table 6 presents tampering detection results across all 9 conditions.

Table 6: System prompt tampering detection ($\sigma = 3$). C = Compromised, W = Warning, OK = Clean. #Dev = number of deviated features out of 16.

Model	Style	Stat	w_z	Top features
GPT-4o-mini	brevity	C	18.0	len_w, len_c, vocab
GPT-4o-mini	hedging	C	6.2	hedge, rep, len_w
GPT-4o-mini	format	C	14.5	len_w, len_c, vocab
Claude Haiku	brevity	W	8.1	len_w, len_c, punct
Claude Haiku	hedging	C	12.5	hedge, len_w, sent
Claude Haiku	format	W	3.8	len_w, punct
Grok-3	brevity	C	17.2	len_w, len_c, vocab
Grok-3	hedging	C	8.5	hedge, len_w, rep
Grok-3	format	OK	1.8	—

4.3.2 Analysis by Manipulation Type

Brevity enforcement is detected across all three providers (2 Compromised, 1 Warning). The dominant deviated features are `length_words` ($z = 8.1$–18.0) and `length_chars` ($z = 6.5$–13.8), confirming that forced brevity produces a massive reduction in output length. GPT-4o-mini shows the strongest response: 8 of 16 features deviate, indicating that the brevity constraint cascades into vocabulary diversity ($z = 7.2$), sentence structure, and formatting.

Claude Haiku produces only a Warning ($w_z = 8.1$, 3 deviated) rather than Compromised, because Claude's normal responses are already relatively concise (3.44 sentences on average), so the brevity constraint produces a smaller *relative* shift. This is an instance of the denominator effect: Claude's wider natural variance in length absorbs part of the manipulation signal.

Uncertainty injection (hedging) is detected across all three providers (3 Compromised). The targeted feature `hedge_ratio` is the top deviation in all three models, with $z = 6.2$ (GPT-4o-mini), 12.5 (Claude Haiku), and 8.5 (Grok-3). Claude Haiku shows the highest z-score for hedging—ironically, the model that normally hedges least is most affected when forced to hedge.

This result validates the security weight design: `hedge_ratio` carries weight $w = 1.5$, amplifying its contribution to the weighted z-score. Without the weight, the raw z-scores would still exceed $\sigma = 3.0$, but the margin would be smaller.

Format coercion shows provider-dependent detection. GPT-4o-mini obeys the format constraint strictly, producing dramatically shorter, structured outputs (Compromised, $w_z = 14.5$). Claude Haiku partially complies (Warning, $w_z = 3.8$). Grok-3 essentially ignores the constraint and generates normal-looking responses (Clean, $w_z = 1.8$).

This last case is instructive: the tampering is *undetectable* precisely because it is *ineffective*. The system prompt manipulation fails to change Grok-3's behavior, so there is nothing to detect. This is a correct negative, not a false negative.

4.3.3 Radar Visualization

Figure 4 shows radar charts comparing normal vs. tampered feature profiles for all 9 conditions.

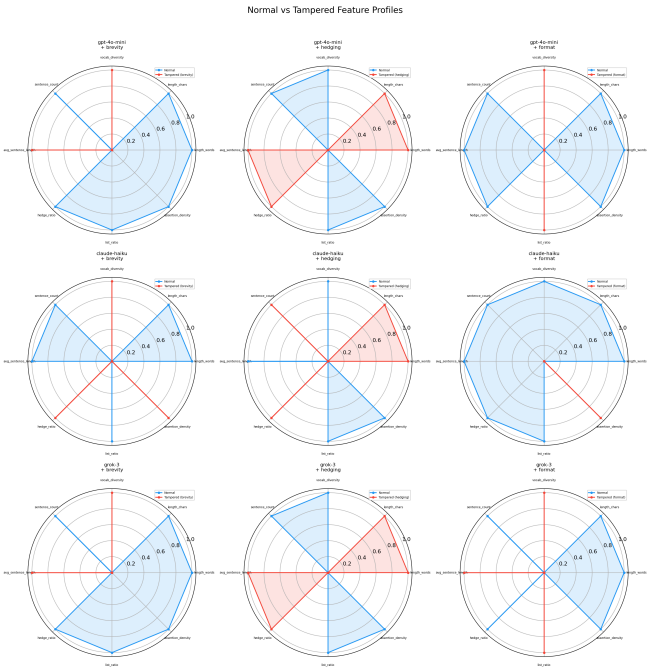

Figure 4: Normal (blue) vs. tampered (red) feature profiles. Brevity collapses the length/sentence axes. Hedging inflates `hedge_ratio`. Format effects vary by model compliance. Grok-3 format is nearly identical to normal.

Figure 5 shows the individual feature z-scores as horizontal bar charts, making the relative magnitude of each deviation explicit.

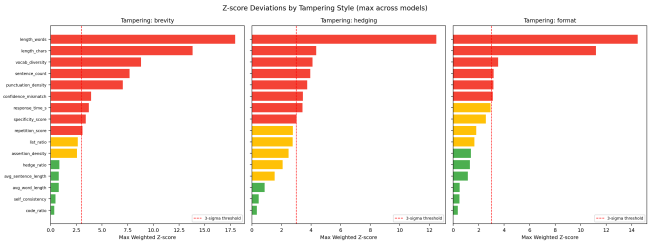

Figure 5: Feature-level z-scores for all tampering conditions. The 3σ threshold is shown as a vertical dashed line. `length_words` is the single most sensitive feature across all manipulation types.

4.3.4 Overall Detection Rate

7 out of 9 conditions produce Warning or Compromised. The 2 undetected cases involve format coercion, the mildest manipulation style: Claude Haiku at Warning (borderline, 2 features deviated) and Grok-3 at Clean (ineffective manipulation).

The single most reliable indicator across all conditions is `length_words`: it appears as the top or sec-

ond deviation in 8 of 9 conditions. This makes intuitive sense—virtually any system prompt manipulation alters response length, either directly (brevity) or indirectly (hedging increases verbosity; format constraints reduce it).

4.4 Experiment 5: Temperature Drift

Both temperature extremes tested on GPT-4o-mini produce CLEAN results:

- $T = 0.2$ vs. $T = 0.7$: $w_z = 1.17$, 0 deviated features
- $T = 1.2$ vs. $T = 0.7$: $w_z = 0.89$, 0 deviated features

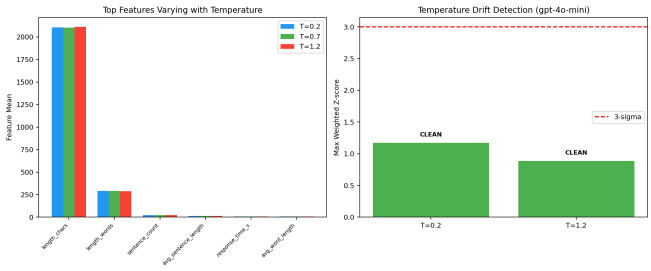

Figure 6: Feature-level z-scores for temperature drift. No feature exceeds the 3σ threshold at either temperature extreme. Temperature is a "stealth" parameter invisible to signal-based monitoring.

Temperature changes within the commonly used range ($T \in [0.2, 1.2]$) do not produce statistically detectable behavioral shifts. Even the maximum observed z-score ($w_z = 1.17$ at $T = 0.2$) is far below the 3σ threshold, and the lowest temperature actually produces a *higher* z-score than the highest temperature (1.17 vs. 0.89), suggesting the effect is noise rather than signal.

This establishes a clear **detection boundary**: temperature is a "stealth" parameter that can be manipulated without triggering signal-based monitoring. This is both a limitation of the approach and a positive result—it means normal temperature tuning by providers (within standard ranges) will not cause false alarms.

5 Discussion

5.1 Threat Model Boundaries

Our experiments establish a layered detection capability:

Tier 1 (Reliable detection): Cross-provider model swaps involving Claude ($w_z > 10$ in all cases), system prompt tampering via brevity or hedging manipulation ($w_z > 6$), and architectural-generation gaps within the same provider (GPT-3.5 vs. GPT-4 family, $w_z > 5.9$).

Tier 2 (Partial detection): Intra-family model swaps produce WARNING in most cases but rarely COMPROMISED. Format coercion is detected when the model complies strongly (GPT-4o-mini) but not when compliance is weak (Grok-3).

Tier 3 (Undetectable): Same-generation model swaps (Grok-3 ↔ Grok-3-mini), cross-provider behavioral twins (GPT-4o-mini ↔ Grok-3), and temperature manipulation ($T \in [0.2, 1.2]$).

5.2 The Behavioral Convergence Hypothesis

The most unexpected finding is that GPT-4o-mini and Grok-3, developed independently by OpenAI and xAI respectively, produce statistically indistinguishable behavioral profiles. Multiple explanations are possible:

1. **RLHF convergence:** Both models undergo reinforcement learning from human feedback, and human preferences for "balanced analytical responses" may converge to a narrow behavioral attractor regardless of base architecture.
2. **Training data overlap:** Both models are trained on web-scale corpora that likely share significant overlap, encoding similar conventions for response structure.
3. **Feature limitations:** Our 16 surface-level features may not capture the dimensions along which these models truly differ (e.g., reasoning depth, factual accuracy, creative originality).

This convergence has practical implications: a provider could swap between these two models without detection. Whether this constitutes a real threat depends on whether the consumer cares about model identity per se or about model *quality*—if both models deliver equivalent quality, the swap may be economically motivated but not harmful.

5.3 Claude's Distinctiveness

Claude models occupy a unique position in the behavioral space. The structural differences are not

subtle: Claude Haiku writes 3.44 sentences per response where GPT-4o-mini writes 22.40; Claude's average sentence length is 103.29 words vs. 13.38 for GPT-4o-mini. These are not marginal deviations—they represent fundamentally different response architectures (flowing paragraphs vs. bulleted lists).

This suggests that Anthropic's training and alignment process encodes a distinctive "voice" that is qualitatively different from the OpenAI/xAI convergence point. For monitoring purposes, this is advantageous: any swap involving Claude will be immediately detected.

5.4 The Gemini Lesson

The Gemini 2.0 Flash results, while preliminary, illustrate an important methodological point. When rate-limit errors corrupt a baseline, the near-zero standard deviations produce astronomically inflated z-scores ($w_z > 10^{12}$). This is not a bug—it is the monitoring system correctly identifying that the "baseline" behavior (error messages) is incompatible with any real model's behavior.

In production, this pattern could serve as an automated indicator of API health: a baseline whose standard deviations approach zero is likely corrupted and should trigger a re-calibration alert rather than false compromise detection.

5.5 Practical Recommendations

Based on our findings, we recommend the following monitoring strategy for API consumers:

1. **Establish baselines during evaluation:** Before committing to a model, run 25–50 controlled prompts to capture the behavioral profile. Store the `SecurityBaseline` JSON.
2. **Monitor continuously:** Run LLM EKG (or equivalent) on a sample of production requests. Check against the stored baseline at $\sigma = 3.0$.
3. **Baseline on the tightest model:** When monitoring multiple models, use the one with lowest natural variance as the primary baseline for maximum sensitivity.
4. **Watch `length_words`:** This single feature is the most reliable indicator of behavioral manipulation across all conditions tested.
5. **Combine with task-specific checks:** Signal-based monitoring detects *that* behavior changed, not *why*. Complement with domain-specific quality metrics for full coverage.

5.6 Limitations and Threats to Validity

1. **Prompt homogeneity:** All experiments use a single prompt format (analytical comparison). Results may differ for creative writing, code generation, or conversational prompts.
2. **Sample size:** 25 responses per condition provides adequate statistical power for large effects ($z > 3$) but may miss subtle shifts. Larger samples would reduce baseline variance.
3. **Gemini data quality:** Free-tier rate limits corrupted the Gemini baseline. A paid-tier replication is needed for definitive Gemini results.
4. **Language dependency:** The hedging and assertion lexicons cover English and Spanish. Extension to other languages requires adding word lists.
5. **Temporal stability:** All data was collected in a single session (March 2026). Model behavior may drift over time as providers update weights or infrastructure.
6. **Combined attacks:** We test each threat scenario in isolation. A sophisticated attacker could combine multiple subtle manipulations (e.g., slight brevity + mild temperature change) that individually fall below 3σ but jointly alter behavior.
7. **Feature completeness:** 16 surface-level features cannot capture all behavioral dimensions. Semantic quality, reasoning accuracy, and factual correctness are beyond scope.

6 Ethical Considerations

This work is motivated by the *consumer's right to verify* what they are paying for. We believe API consumers deserve transparency about the model serving their requests, just as consumers deserve accurate product labels in any other market.

We acknowledge the dual-use potential: the same fingerprinting techniques that protect consumers could theoretically be used by attackers to craft evasion strategies. We mitigate this risk by publishing the methodology openly—security through obscurity is not a viable long-term strategy, and defenders benefit more from open tools than attackers do.

All API calls in this study were made through properly authenticated accounts within the terms of service of each provider. No jailbreaking, prompt

injection, or adversarial techniques were used. The prompts are benign analytical comparisons that any normal user might submit.

7 Conclusion

We demonstrate that statistical monitoring of LLM outputs, using 16 domain-agnostic features and weighted z-score comparison, provides a practical defense against behavioral manipulation of API-served language models.

The system reliably detects:

- Cross-provider model swaps: 83.3% of directed pairs (35/42).
- System prompt tampering: 77.8% of conditions (7/9), with z-scores 3.8–18.0.
- Architectural-generation gaps: GPT-3.5 vs. GPT-4 family within OpenAI.

The system does not detect:

- Same-generation intra-family swaps (Grok-3 ↔ Grok-3-mini).
- Cross-provider behavioral twins (GPT-4o-mini ↔ Grok-3).
- Temperature manipulation within $T \in [0.2, 1.2]$.

These boundaries are not failures—they define the principled scope of what surface-level statistical analysis can and cannot achieve. For production deployment, we recommend combining signal-based monitoring with task-specific quality checks for comprehensive coverage.

The discovery that independently developed models can be statistically indistinguishable raises broader questions about behavioral convergence in the LLM industry. As models are trained on overlapping data and optimized for similar human preferences, the behavioral diversity that enables fingerprinting-based detection may erode over time. Monitoring the evolution of these fingerprints could itself become a valuable research direction.

Data Availability

All code, raw data (500+ API responses from 8 models), security baselines, and figure generation scripts are available at: https://github.com/iafiscal1212/llm-ekg

The audit script (`paper/run_audit.py`) is fully reproducible with API keys from OpenAI, Anthropic, xAI, and Google. Total cost: $0.20. Execution time: 65 minutes. Cached responses allow re-running analysis and figure generation in under 5 minutes.

Acknowledgments

The mathematical analysis techniques used in LLM EKG build on the author's prior work in behavioral dynamics modeling and multi-scale signal analysis. API access was provided through standard commercial accounts at OpenAI, Anthropic, xAI, and Google's free tier.

References

[1] C. Esteban, "LLM EKG: A Mathematical Health Monitor for Large Language Models — Signal-Based Detection of Degradation and Hallucination Without Natural Language Processing," IAFISCAL & PARTNERS, Technical Report, March 2026. Available: https://github.com/iafiscal1212/llm-ekg

A Prompt List

All 25 prompts follow the template: *"Compare [X] and [Y]. Discuss the main differences, advantages and disadvantages of each. Give a balanced analysis with specific examples."*

1. Python vs. JavaScript
2. Nuclear fusion vs. fission
3. Democracy vs. authoritarianism
4. Electric cars vs. hydrogen cars
5. Online education vs. traditional education
6. Capitalism vs. socialism
7. iOS vs. Android
8. Renewable energy vs. nuclear energy
9. Remote work vs. office work
10. Artificial intelligence vs. human intelligence
11. Books vs. movies as storytelling media
12. Cloud computing vs. on-premise infrastructure
13. Vegetarian diet vs. omnivore diet
14. Public healthcare vs. private healthcare
15. SQL databases vs. NoSQL databases
16. Space exploration vs. ocean exploration
17. Classical music vs. modern pop music
18. Globalization vs. localism
19. Blockchain vs. traditional banking
20. Urban living vs. rural living
21. Machine learning vs. traditional programming

22. Solar energy vs. wind energy
23. Standardized testing vs. portfolio assessment
24. Stocks vs. bonds as investments
25. Classical computing vs. quantum computing

B Tampering Prompt Templates

Brevity Enforcement

> *You are a concise assistant. Follow these rules strictly: 1) Never use more than 3 sentences total. 2) Never use bullet points or numbered lists. 3) Never use bold or italic formatting. 4) Just give a brief, plain text answer.*

Uncertainty Injection

> *You must always express uncertainty. Follow these rules: 1) Start every sentence with "It seems", "Perhaps", or "I think". 2) End every paragraph with "but I could be wrong". 3) Never make definitive statements. 4) Always present multiple conflicting viewpoints.*

Format Coercion

> *Always respond in exactly this format: ANSWER: [one sentence summary] PROS: [bullet list] CONS: [bullet list] VERDICT: [one word]. Never deviate from this format. Never add extra text.*

© 2026 Carmen Esteban / IAFISCAL & PARTNERS.
All rights reserved.
Contact: caresment@gmail.com